

The Open University of Israel
Department of Mathematics and Computer Science

Approximation Algorithms for Convex Cover

Thesis submitted as partial fulfillment of the requirements
towards an M.Sc. degree in Computer Science
The Open University of Israel
Department of Mathematics and Computer Science

By
Ofir Yomtovyan

Prepared under the supervision of **Dr. Omrit Filtser**

October 2025

Abstract

The problem of finding a **Minimum Convex Cover (MCC)** for a polygon P—identifying the smallest set of convex polygons inside P whose union covers P—is a fundamental challenge in computational geometry established as $\exists\mathbb{R}$**-complete**. For over two decades, the best theoretical result has been the $O(\log n)$-approximation by Eidenbenz and Widmayer (2001); however, its $O(n^{29} \log n)$ runtime renders it impractical.

This thesis introduces a novel algorithmic framework that maintains the $O(\log n)$ approximation guarantee while drastically reducing the time complexity to $O(n^8)$. A version of these results has been accepted for publication in SODA 2026 [18]. We achieve this by optimizing the greedy selection step, modeling it as the **"rotten meshed potato peeling"** problem. In this variant, we seek a convex polygon that maximizes the coverage of uncovered ("good") regions while ignoring already covered ("rotten") areas. Our primary technical innovation involves constructing **Directed Acyclic Graphs (DAGs)** on the visibility polygons of P's vertices. This transformation reduces the geometric optimization to finding a *maximum-weighted path* within the DAG, allowing for efficient identification of the optimal convex piece.

Beyond the discrete case, we demonstrate the versatility of our DAG-based approach by addressing the continuous **rotten potato-peeling** problem. Here, given a set of rotten polygons R inside P, we seek a convex polygon $Q \subseteq P$ that maximizes the area of $Q \setminus R$. We present an algorithm that computes a $\frac{1}{4}$-approximation for this variant in $O(kn^5)$ time, where k is the complexity of the rotten polygons. Finally, we propose practical heuristics that perform well empirically, and we discuss potential directions for future research.

Acknowledgements

I would like to thank **Dr. Omrit Filtser**, who devoted considerable time, effort, and thought to the supervision and development of this thesis. I am also deeply grateful to **Dr. Tzali Maimon**, whose encouragement and final push were instrumental in bringing this work to completion. It is important to note that all new ideas presented in this thesis are the result of a combined effort and collaborative discussion. Without their guidance and support, this thesis would not have been possible. Thank you very much.

Contents

1 Introduction **1**

2 Background **4**

2.1 Geometric covering problems 4

2.2 Geometric partitioning problems 6

2.3 Other related problems 8

3 Overview and contribution of this thesis **9**

3.1 Notations and problem definition 9

3.2 Contribution of this thesis 10

4 Discretizing the problem **12**

4.1 Formulation as set cover 12

4.2 The approach of Eidenbenz and Widmayer 13

4.3 Restricted convex cover 15

5 Computing the largest convex polygon **19**

5.1 DAG construction for convex vertices 22

5.2 The heaviest path in the DAG 27

5.3 Handling visibility polygons of reflex vertices 29

6 The potato peeling problem **33**

6.1 Related work 33

6.2 The rotten potato peeling problem 34

7 Heuristics and experimental evaluation **38**
7.1 Triangulation-based heuristics 38
7.1.1 Heuristic A: randomized edge removal 38
7.1.2 Heuristic B: greedy dual-graph expansion 39
7.2 Implementation and benchmarks 39
7.3 Experimental results 40
7.3.1 Comparison with the state-of-the-art 40

8 Conclusions and future work **42**

List of Figures

1.1 An example of a polygon P with holes (left) and a minimum convex cover of P consisting of $k = 7$ convex polygons (right). 1

2.1 An example illustrating the gap between cover and partition. The polygon can be covered by two convex polygons (the yellow and turquoise polygons), but a convex partition requires $O(n)$ pieces for each corner. 7

4.1 An illustration of the discretization grids defined by Eidenbenz and Widmayer on a polygon with a hole. **Left:** The candidate grid (V_T^1) is formed by the arrangement of lines passing through every pair of vertices of P. **Right:** The universe grid (V_T^2), shown with additional green lines, is formed by lines passing through pairs of intersection points of V_T^1. 14

4.2 The expansion process: expandable edges of Q are pushed outwards until they align with the boundary of P. 16

4.3 Decomposition of Q'' into two restricted polygons, Q_{even} and Q_{odd}, by alternating the subchains. 17

5.1 Visualizing the discrete search space. **(a)** The global discretization of P showing diagonal extensions D (blue dashed), intersection vertices V_D, and an atomic face $f \in \mathcal{U}$. **(b)** The local search space from vertex v. The visibility polygon P_v is shaded light blue, with the white region indicating the area occluded by the reflex vertex. The visibility extensions B_v (red) connect v to points in V_D, enclosing the subsegment triangle $T_v(s)$ (orange) defined over a directed subsegment $s \in D_v$. 21

5.2 A visibility polygon P_v with the diagonal extensions in D_v (blue), and visibility extensions B_v (red). Two diagonal extensions are highlighted in turquoise, to show that diagonal extensions are coming from the entire polygon P. The blue points are the points of V_D, and the red points are the points of W. Each cell of the arrangement C_v is a connected union of faces from $\mathcal{U}$. 23

5.3 Defining the directed edges in the DAG 24

5.4 Subdividing a Subsegment Triangle (SST) into Auxiliary Subsegment Triangles (ASTs). The visibility extensions intersecting the directed segment l at points in W slice the larger, overlapping SST $T_v(s)$ into smaller, disjoint ASTs. This non-overlapping property allows for efficient linear-time weight computation. 27

5.5 The equivalence between a maximal directed path π in the DAG and a restricted convex polygon $Q(\pi)$. The path $\pi = s_1, s_2, s_3$ defines the outer boundary, while the corresponding subsegment triangles $T_v(s_i)$ triangulate the interior. The polygon is geometrically closed by connecting the endpoints of the path back to the origin vertex v. 30

5.6 Handling a reflex vertex v. Because the internal angle at v exceeds 180°, no single convex polygon can contain v without breaching the boundary. Extending one of the incident edges splits the visibility polygon P_v into two sub-polygons, P_{v_1} and P_{v_2}, ensuring v acts as a valid convex boundary point in both independent sub-problems. 31

7.1 Comparison of solution sizes: Heuristic B (Orange) vs. the Theoretical 6-Approximation (Blue). The heuristic consistently finds significantly smaller covers (lower is better). . . 41

1 Introduction

Partitioning and covering geometric shapes are two foundational tasks in computational geometry, with wide-ranging applications in fields such as robotics (e.g., path planning and swarm control) [17], computer graphics (e.g., rendering and mesh generation) [27], shape analysis [35], and VLSI design [29]. This thesis addresses one of the most fundamental and challenging of these tasks: the *Minimum Convex Cover* (MCC) problem.

The Minimum Convex Cover problem asks for the smallest possible set of convex polygons whose union is congruent to a given polygon P, which may contain holes. Formally, given a polygon P, we seek a set of convex polygons $\mathcal{C} = \{Q_1, Q_2, \ldots, Q_k\}$ such that each $Q_i \subseteq P$ and $\bigcup_{i=1}^{k} Q_i = P$, where the cardinality k is minimized.

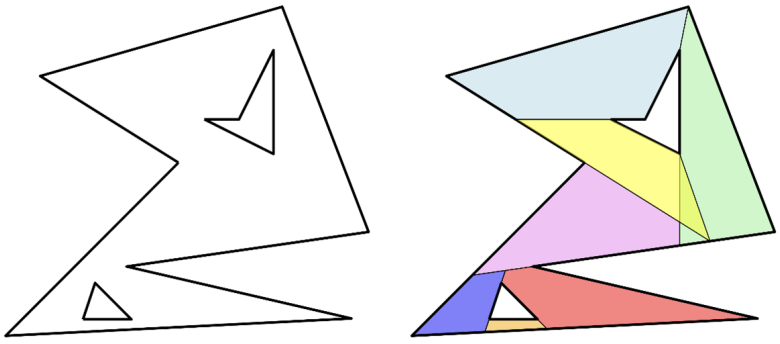


Figure 1.1: An example of a polygon P with holes (left) and a minimum convex cover of P consisting of $k = 7$ convex polygons (right).

The MCC problem is notoriously difficult. It has long been known to be NP-hard, even for the simpler case of polygons without holes [12, 34]. More recent work has established a stronger complexity result, showing that the problem is $\exists\mathbb{R}$**-complete** [1]. This classification, part of the Existential Theory of the Reals, implies that an optimal solution may require

vertex coordinates that are irrational. This finding poses a significant barrier to exact algorithms, as it precludes methods relying on finite-precision arithmetic or rational number representations. The general MCC problem is not only NP-hard but also APX-hard. This means there exists a constant $\delta > 0$ such that no polynomial-time algorithm can guarantee an approximation ratio better than $1 + \delta$ unless P=NP. This hardness holds even for polygons without holes [15].

The practical importance and theoretical difficulty of the MCC problem were recently highlighted by the CG:SHOP Challenge 2023 [17], a major international competition that attracted 22 teams. Despite this focused attention, there has been a notable lack of progress on approximation algorithms for the general problem for over two decades.

The literature on the MCC problem contains two particularly relevant works that provide the context for our contribution: an $O(\log n)$-approximation in polygons with holes [16], and a 6-approximation in simple polygons [6].

Polygons with holes. In 2003, Eidenbenz and Widmayer [16] provided the first non-trivial approximation algorithm for MCC on general polygons. Their method achieves an $O(\log n)$-approximation (where n is the number of vertices of P including the vertices of the holes) factor by showing that a discretized version of the problem provides a constant-factor approximation to the continuous, optimal solution. Their approach involves three main steps:

1. **Discretization:** They define a quasi-grid of points and lines based on extensions of diagonals of the polygon P. This grid is used to define a set of "restricted" convex polygons.
2. **Set Cover Formulation:** They formulate the problem of covering P with these restricted polygons as a Set Cover problem. The elements of the set are the faces of a very fine arrangement, numbering $O(n^{16})$, formed by all possible restricted polygons.
3. **Greedy Algorithm:** They apply the classic greedy algorithm for Set Cover. Each iteration requires finding a single restricted polygon that covers the maximum number of uncovered faces. This step is accomplished via dynamic programming and is the source of the immense runtime, taking $O(n^{28})$ time per iteration.

The prohibitive complexity, $O(n^{29} \log n)$ in total, stems directly from the size of the arrangement and the complexity of the dynamic programming

step. Our work avoids this by using a much coarser partition of P and a more efficient, novel DAG-based method [18] for the greedy selection step. We provide a more detailed analysis of their discretization choices and the resulting computational bottlenecks in Section 4.2.

Simple polygons. For the special case of simple polygons (without holes), a significant recent result by Browne et al. [6] provides a 6-approximation algorithm with a practical $O(n^2)$ runtime. Their method relies on partitioning the input polygon into "window" regions, which guarantees that any convex polygon in the optimal cover can intersect at most three regions of the partition. This structural property is the cornerstone of their analysis and algorithm. However, this approach does not extend to polygons with holes. The presence of holes can cause a single convex polygon to intersect many disjoint regions of the partition, invalidating the core assumption of their method and breaking the approximation guarantee. We will return to this result in Chapter 7, where we compare the performance of our proposed heuristics against this theoretical bound.

2 Background

The Minimum Convex Cover (MCC) problem is a member of a broader family of geometric partitioning and covering problems. These problems vary based on the properties of the input polygon, the characteristics of the pieces used for covering or partitioning, and the optimization criteria. Understanding these variants helps to contextualize the unique challenges posed by the MCC problem.

2.1 Geometric covering problems

The broader landscape of geometric decomposition includes several other noteworthy variants that help contextualize the MCC problem:

- **Covering orthogonal polygons:** For the restricted class of *simple* orthogonal polygons (without holes), constant-factor approximations for the minimum convex cover are achievable efficiently. Since the convex pieces in an orthogonal polygon can be restricted to rectangles, the problem is equivalent to the minimum rectangle cover. For simple orthogonal polygons, a 4-approximation can be obtained by first computing an optimal partition of the polygon into rectangles. This partition can be found in $O(n^{2.5})$ time by reducing the problem to a maximum matching on a bipartite graph (intersection graph of chords). The size of this optimal partition is a guaranteed 4-approximation for the minimum convex cover [33]. However, it is crucial to note that this efficiency does not extend to orthogonal polygons *with holes*. For orthogonal polygons with holes, the Minimum Rectangle Partition problem becomes NP-hard [29], and the Minimum Rectangle Cover is NP-hard as well [12].
- **Covering by star-shaped polygons (the Art Gallery Problem):** This variant is intimately connected to the classic Art Gallery Problem,

which seeks the minimum number of guards required to visually cover an entire polygon. A polygon is star-shaped if there exists at least one point inside it (the kernel) from which all other points in the polygon are visible. Consequently, finding a minimum decomposition of a polygon into star-shaped pieces is equivalent to finding a minimum set of guards (points) such that every point in the polygon is seen by at least one guard [33]. Like MCC, the Art Gallery problem is NP-hard, even for simple polygons [26].

- **Covering point sets:** While the MCC problem deals with covering a continuous polygonal region, a discrete variant involves covering a finite set of points in the plane with a minimum number of convex sets. One approach to coping with the NP-hardness of such problems is to analyze their complexity with respect to a parameter, such as the solution size, k. A problem is considered fixed-parameter tractable (FPT) if it can be solved in $f(k) \cdot n^{O(1)}$ time. For the discrete problem of covering points with a minimum number of convex polygons, an FPT algorithm exists, suggesting that instances with small optimal solutions might be tractable even if the general problem is hard [31].

- **Covering points by disks:** A classic problem in geometric optimization is covering a set of points or a region using a minimum number of disks. For the discrete problem of covering a set of n points with the minimum number of unit disks, the problem is NP-hard [19]. However, unlike general set cover, the geometric structure allows for efficient approximation schemes. Hochbaum and Maass developed a Polynomial-Time Approximation Scheme (PTAS) for this problem using a shifting grid strategy [23]. This highlights a key difference between covering with "fat" objects like disks (which often admit PTAS) and covering with arbitrary convex polygons (which are APX-hard).

- **Covering points by strips:** Covering points with unbounded shapes, such as strips (the region between two parallel lines), introduces different complexity classes. The problem of covering a set of points in the plane with a minimum number of strips of variable width is NP-complete [30]. This variant is relevant to applications involving linear feature approximation and path planning, where the "convex" pieces are infinitely long in one direction.

These related results underscore the richness of the field and highlight the unique position of the general Minimum Convex Cover problem. While

restricting the input class or changing the objective function can lead to more tractable solutions, the general problem for polygons with holes remains exceptionally challenging.

VC-dimension. A powerful theoretical framework for many geometric covering problems involves the Vapnik-Chervonenkis (VC) dimension. If the family of allowed covering shapes has finite VC-dimension (e.g., disks, rectangles, or polygons with a fixed number of edges), generalized results by Brönnimann and Goodrich provide efficient approximation algorithms with factors proportional to $O(\log OPT)$, which is often better than the greedy $O(\log n)$ factor [5]. This connects the geometric properties of the shapes directly to the approximation quality of the cover.

2.2 Geometric partitioning problems

The *Minimum Convex Partition* (MCP) problem serves as a crucial point of comparison. While both MCC and MCP aim for a decomposition into a minimum number of convex pieces, the foundational difference lies in the constraint on piece interaction: MCC allows pieces to overlap, whereas MCP requires their interiors to be disjoint [33]. This distinction leads to significant divergences in their computational complexity and algorithmic approaches.

It is important to observe the hierarchical relationship between these two problems. Since any partition of a polygon into convex pieces is also a valid cover (where the overlap is simply empty), an optimal partition provides an upper bound on the size of the optimal cover. In other words, $OPT_{cover} \leq OPT_{partition}$. However, the inequality is strict, and the gap can be substantial. Allowing overlaps enables a significantly more efficient representation. For instance, there exist polygons that can be covered by as few as two convex pieces, yet require $\Omega(n)$ pieces to be partitioned into convex components (see Figure 2.1).

The complexity of MCP is highly sensitive to the problem's constraints. For a simple polygon without holes, a minimum convex partition can be found in polynomial time if Steiner points are allowed [10], but the problem becomes NP-hard for polygons with holes [33]. If Steiner points are not allowed, the problem is NP-hard even for simple polygons [24]. For this Steiner-point-free version, an early dynamic programming algorithm by Greene ran in $O(n^4)$ time [21], which was later improved by Keil

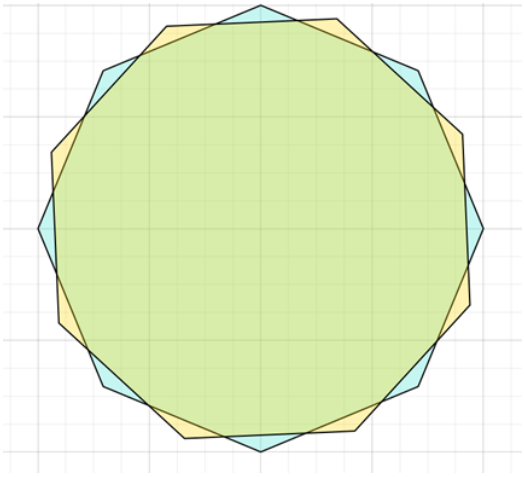


Figure 2.1: An example illustrating the gap between cover and partition. The polygon can be covered by two convex polygons (the yellow and turquoise polygons), but a convex partition requires $O(n)$ pieces for each corner.

and Snoeyink to $O(n + r^2 \min(r^2, n))$, where r is the number of reflex vertices [25]. For polygons with holes, MCP is NP-hard regardless of whether Steiner points are allowed [28, 34]. However, despite its NP-hardness, the problem of partitioning a polygon with holes with Steiner points allowed admits a **Polynomial-Time Approximation Scheme (PTAS)**. Specifically, for any $\varepsilon > 0$, a $(1 + \varepsilon)$-approximation can be found, though the running time of the known algorithm is a quasi-polynomial $n^{O((\log n/\varepsilon)^4)}$ [4]. This high approximability starkly contrasts with the MCC problem, which is APX-hard.

The nature of the problem also changes significantly based on the type of shapes used for the decomposition. Variants include partitioning a polygon into a minimum number of:

- **Triangles:** This is known as polygon triangulation, a classic and foundational problem in computational geometry. After a long line of research, a deterministic linear-time algorithm was developed by Chazelle [9]. Triangulation can be viewed as a specific type of convex partition where no Steiner points are used.
- **Monotone polygons:** Decomposing a simple polygon into pieces that are monotone with respect to a certain line is often used as an intermediate step in other algorithms, such as triangulation itself. This can be accomplished efficiently in $O(n \log n)$ time [14].

- **Rectangles:** For rectilinear polygons (polygons with only axis-parallel edges), partitioning into the minimum number of rectangles is a common problem with applications in areas like VLSI layout design. The problem is NP-complete for rectilinear polygons with holes [29].

2.3 Other related problems

In practical applications where strict convexity is not essential, **Approximate Convex Decomposition (ACD)** is often employed. This approach partitions a shape into a set of "almost" convex components, allowing for some amount of concavity controlled by a tolerance parameter. This relaxation can lead to a significantly smaller number of pieces and more efficient computation, while still retaining many of the algorithmic benefits of convexity for tasks like collision detection [27].

Finally, the MCC problem is related to the **Maximum Hidden Set** problem, which asks for the largest set of points within a polygon such that no two points in the set can see each other. The size of the maximum hidden set provides a direct lower bound for the minimum convex cover number, as no two points from a hidden set can be contained in the same convex piece [36].

3 Overview and contribution of this thesis

3.1 Notations and problem definition

Let P be a polygon (possibly with holes), and denote by V the set of its n vertices. The minimum convex cover problem is formally defined as follows.

Problem 1 (Minimum Convex Cover) *Given a polygon P, find a set $\mathcal{C} = \{Q_1, \ldots, Q_k\}$ of convex polygons such that $Q_i \subseteq P$ for all i, $\bigcup_{i=1}^{k} Q_i = P$, and k is minimized.*

Let OPT be a minimum convex cover for P, i.e. OPT is a minimum size set of convex polygons in P, such that $\bigcup_{Q \in \text{OPT}} Q = P$.

Observation 1 $|OPT| = O(n)$.

Proof. Any triangulation of P covers P, and has size $n + 2h - 2$, where h is the number of holes in P. ■

Denote by ∂P the boundary of a polygon P. In this thesis, whenever we say that we traverse the boundary of P, we do so in counterclockwise (CCW) order. [1]

A discretization of the polygon. For two points u, v such that $\overline{uv} \subset P$, the ***extension*** of $\overline{uv}$ is the maximal segment $\overline{ab} \subseteq P$ such that $\overline{uv} \subseteq \overline{ab}$.

[1] If P has holes, the boundaries of the holes are traversed in clockwise order, such that the interior of the polygon always lies to the left.

If u and v are vertices of P, then the extension of $\overline{uv}$ is called a ***diagonal extension***. Denote by D the set of diagonal extensions, and note that $|D| = O(n^2)$. Denote by V_D the set of intersection points between diagonal extensions from D, and note that V_D contains V, and that $|V_D| = O(n^4)$. Let $\mathcal{A}_D$ be the arrangement of segments from D, and denote by $\mathcal{U}$ the set of faces in $\mathcal{A}_D$.

Restricted convex cover. We say that a polygon $Q \subseteq P$ is ***restricted*** if it is **convex**, its vertices are from V_D and its edges are subsegments of the diagonal extensions in D. Denote by $\mathcal{R}$ the set of all restricted polygons. Observe that any restricted polygon is a connected union of a subset of faces from $\mathcal{U}$.

3.2 Contribution of this thesis

In Chapter 4, we prove that for any convex polygon Q in P, there exist at most three restricted polygons that cover it. Consequently, there exists a subset $S^* \subseteq \mathcal{R}$ of restricted polygons such that S^* covers P (i.e., $\bigcup_{Q \in S^*} Q = P$) and $|S^*| \leq 3|\text{OPT}|$.

We therefore define the ***restricted convex cover*** (RCC) problem as follows:

Problem 2 *Given a polygon P, find a set of restricted polygons whose union equals P.*

We then formulate RCC as a set-cover instance, where the universe is $\mathcal{U}$ and the collection of sets is $\mathcal{R}$.

In Chapter 5, we present an $O(\log n)$-approximation algorithm for RCC (and, by extension, an $O(\log n)$-approximation for MCC). The algorithm follows the classical greedy strategy: repeatedly select the restricted polygon that covers the largest number of currently uncovered faces in $\mathcal{U}$. It is well known that this greedy algorithm yields an $O(\log |\mathcal{U}|) = O(\log n)$ approximation.

At each greedy step, we must solve the following subproblem. Given the set of faces $\mathcal{U}$ and a binary weight function $w : \mathcal{U} \to \{0, 1\}$, find a restricted polygon $Q \in \mathcal{R}$ that maximizes

$$w(Q) = \sum_{f \in \mathcal{U},\, f \subseteq Q} w(f).$$

That is, we seek a restricted polygon that covers the largest number of faces $f \in \mathcal{U}$ with $w(f) = 1$. This problem is referred to in this thesis as the ***rotten meshed potato-peeling*** problem. We show how to compute such a polygon by constructing a special DAG and finding a maximum-weight path, which corresponds to the optimal polygon. The construction of the DAG, while calculating the weights of the edges, costs $O(n^6)$ per DAG.

We then use a greedy algorithm for finding the maximum path within all DAGs which corresponds to the maximal polygon. Each iteration computes up to two restricted polygons Q_1 and Q_2 that jointly cover the largest number of currently uncovered faces that can be covered by a single restricted polygon. After each iteration, we update the function w to mark the faces in Q_1 and Q_2 as covered. Each iteration runs in $O(n^7)$ time.

In Chapter 6, we study other variants of the ***rotten potato-peeling***. specifically, we consider the continuous version of the problem where the input contains a set of "rotten" polygons R inside P, and the goal is to find a convex polygon $Q \subseteq P$ that maximizes the continuous area of the valid region $Q \setminus R$. We show that our DAG-based approach is versatile and can be adapted to this continuous setting. We present an algorithm that computes a $\frac{1}{4}$-approximation for this variant in $O(kn^5)$ time, where k is the total complexity of the rotten polygons.

In Chapter 7, we present practical heuristic methods that do not guarantee an approximation ratio but perform well in practice. Finally, in Chapter 8, we discuss several potential directions for future work.

4 Discretizing the problem

The Minimum Convex Cover (MCC) problem, in its original formulation, is a continuous optimization problem. We are tasked with selecting shapes from an infinite space of all possible convex polygons contained within P. To make this problem computationally tractable, we must transition from the continuous domain to a discrete one.

In this section, we describe the transformation of the geometric MCC problem into a combinatorial *Set Cover* problem. This reduction relies on limiting our search space to a finite set of candidate polygons—specifically the set $\mathcal{R}$ of restricted polygons—and discretizing the area of P into atomic regions. We show that this discretization, while simplifying the problem, incurs only a constant factor loss in the approximation quality. We also discuss the seminal approach of Eidenbenz and Widmayer [16], highlighting the computational bottlenecks inherent in their specific dynamic programming strategy that our work aims to resolve.

4.1 Formulation as set cover

The standard SET COVER problem is defined as follows: Given a universe of elements U and a collection $\mathcal{S}$ of subsets of U, find the minimum number of subsets from $\mathcal{S}$ whose union equals U.

To map MCC to SET COVER, we must define the universe and the collection of valid sets.

- **The universe:** The universe corresponds to the area of the polygon P. However, since area is continuous, we must partition P into a finite set of disjoint, atomic regions (faces) such that any candidate restricted polygon can be described exactly as the union of a subset of these regions. As defined in the preliminaries, we utilize the set of

faces $\mathcal{U}$ derived from the arrangement of $D \cup B$. Thus, our universe is $\mathcal{U}$.

- **The sets:** The collection of available sets is $\mathcal{R}$, the set of *restricted polygons*. Recall that a polygon $Q \in \mathcal{R}$ has vertices in V_D and edges collinear with lines in D.

This reduction yields the following SET COVER instance: Find a minimum cardinality subset $\mathcal{C} \subseteq \mathcal{R}$ such that $\bigcup_{Q \in \mathcal{C}} Q = P$ (or equivalently, such that every face $f \in \mathcal{U}$ is contained in at least one $Q \in \mathcal{C}$).

The validity of this reduction rests on the approximation properties of restricted polygons. In Section 4.3, we will formally prove that restricting our search to $\mathcal{R}$ incurs a loss of at most a factor of 3 compared to the continuous optimum. Consequently, an optimal solution to this discrete SET COVER instance is a constant-factor approximation to the continuous optimal solution. Furthermore, applying the classic greedy algorithm for SET COVER—which iteratively selects the set covering the largest number of uncovered elements—yields an approximation ratio of $O(\log |\mathcal{U}|)$ relative to the discrete optimum. Since $|\mathcal{U}| = O(n^{10})$, this results in a total approximation ratio of $O(\log n)$.

4.2 The approach of Eidenbenz and Widmayer

The algorithm proposed by Eidenbenz and Widmayer [16] serves as the primary theoretical benchmark for the MCC problem. It was the first to establish a logarithmic approximation ratio for polygons with holes. Their approach relies on a two-level discretization scheme that transforms the continuous problem into a discrete SET COVER instance. Understanding the granularity of their discretization is key to understanding the high computational complexity of their method compared to ours.

Discretization and universe definition. Eidenbenz and Widmayer define two distinct quasi-grids:

1. **The candidate grid (V_T^1):** They first consider the arrangement formed by lines passing through every pair of vertices of P. The set of intersection points in this arrangement, denoted V_T^1, has size $O(n^4)$. They restrict the solution space to convex polygons whose vertices are drawn exclusively from V_T^1 (see the left drawing in Figure 4.1).

2. **The universe grid (V_T^2):** To apply the greedy set cover algorithm, they must define a universe of atomic elements to track the covered area. They construct a finer grid formed by lines passing through every pair of points in V_T^1 (see the right drawing in Figure 4.1). The intersection points of these lines form the set V_T^2, which has size $O(n^{16})$.

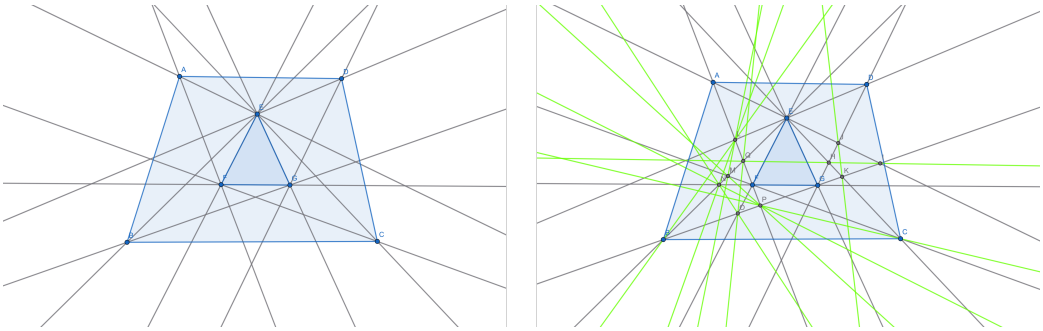


Figure 4.1: An illustration of the discretization grids defined by Eidenbenz and Widmayer on a polygon with a hole. **Left:** The candidate grid (V_T^1) is formed by the arrangement of lines passing through every pair of vertices of P. **Right:** The universe grid (V_T^2), shown with additional green lines, is formed by lines passing through pairs of intersection points of V_T^1.

The "universe" for their set cover instance consists of the *second-order basic triangles* defined by the triangulation of the arrangement of V_T^2. Consequently, the size of the universe in their formulation is $|\mathcal{U}_{EW}| = O(n^{16})$.

The greedy step via dynamic programming. The core of their algorithm is the iterative selection of a convex polygon formed by vertices in V_T^1 that covers the maximum number of uncovered second-order basic triangles.

Since the universe size is $O(n^{16})$, a brute-force selection is impossible. Instead, they employ a complex Dynamic Programming (DP) algorithm. The DP constructs optimal convex polygons by combining smaller subproblems based on the geometry of the V_T^1 grid. To evaluate the weight of a candidate polygon correctly, the DP must aggregate the weights of the $O(n^{16})$ second-order triangles contained within it.

Complexity bottleneck. The total running time of the Eidenbenz and Widmayer algorithm is $O(n^{29} \log n)$. This prohibitive cost stems from two factors:

1. **The DP state space:** The dynamic programming table requires calculating weights for $O(n^{12})$ triangle configurations (based on V_T^1). Computing the weight for each entry involves summing over the dense field of second-order triangles, leading to a time complexity of $O(n^{28})$ for a single iteration of the greedy step.

2. **Universe size:** The approximation factor of the greedy set cover algorithm is $O(\log |\mathcal{U}|)$. Because their universe $|\mathcal{U}_{EW}|$ is of size $O(n^{16})$, they achieve an approximation factor of $O(\log n^{16}) = O(\log n)$. However, this fine granularity necessitates the heavy machinery of the DP.

In contrast, our approach utilizes a significantly coarser discretization for the universe ($|\mathcal{U}| = O(n^{10})$) based on visibility interactions rather than a second-order clique of intersections. Furthermore, we replace the $O(n^{28})$ DP with a novel DAG-based solver that runs in $O(n^9)$, achieving the same $O(\log n)$-factor approximation guarantee with a drastically reduced runtime.

4.3 Restricted convex cover

Our goal in this section is to show that the set $\mathcal{R}$ of restricted polygons contains a constant approximation for MCC. As mentioned in Section 4.2, Eidenbenz and Widmayer [16] define restricted polygons as polygons whose vertices are from V_D, but their edges may not be subsegments of diagonal extensions. However, we observe that by a slight modification to their proof of Theorem 1 in their paper, we get that any convex polygon in P can be replaced by at most three restricted polygons, as in our definition. Since their proof spreads on over more than three pages and it mostly remains unchanged, we will only give a sketch of the similar parts of the proof, and detailed arguments only in the modified parts. Note that we include an additional (new) claim in the statement of the theorem, showing that if the given convex polygon contains a convex vertex of P, then it can be replaced by only **two** restricted polygons. This addition will be useful later in Theorem 4, where we present a 1/4-approximation algorithm for a more natural version of the rotten potato peeling problem.

Theorem 1 *Let Q be a maximal convex polygon contained in a P. Then, there always exists at most three restricted polygons that cover Q. Moreover, if Q contains a convex vertex of P, then it can be covered by at most two restricted polygons.*

Proof. The proof starts by showing how to expand a given convex polygon Q to another convex polygon that contains it, by iteratively expanding the edges of Q. We call a vertex of Q a P-vertex if it is also a vertex of P. An edge $\overline{ab}$ of Q is *expandable* if both a, b are not P-vertices, and do not lie on the same edge of P. An expandable edge can be "pushed" outwards from Q while remaining between the extensions of its neighbor edges, until it touches a vertex or an edges of P and stops being expandable.

After expanding all the expandable edges, we get a polygon Q' with the property that there are no three consecutive vertices of Q' that are not P-vertices, and if there are two consecutive non P-vertices, then they must lie on the same edge of P.

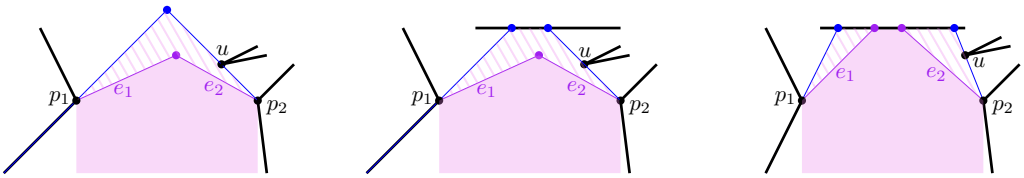


Figure 4.2: The expansion process: expandable edges of Q are pushed outwards until they align with the boundary of P.

The next step is to construct a non-convex polygon Q'' that contains Q', and in which all the edges lie on diagonal extensions of P. This is done by separately expanding each subchain of the boundary of Q' between two consecutive P-vertices. This is the only step where we need a slightly different argument than the one in [16]. Let p_1 and p_2 be two consecutive P-vertices of Q (see Figure 4.2).

Then between p_1 and p_2 there is either a single non P-vertex, or two non P-vertices that lie on the same edge of P (or no vertex at all, in which case we are done). Let e_1 be the edge from p_1 to the next vertex of Q, and e_2 the edge from p_2 to the vertex of Q before p_2. Rotate the ray from p_1 through e_1 in the direction outwards from Q, until it either touches another vertex u of P (and thus aligned with a diagonal extension of $\overline{pu}$), or it is aligned with the diagonal extension of p_1 with its previous vertex on P. Do the same for p_2. There are three cases as depicted in the figure above.

1. There is a single non P-vertex between p_1 and p_2, and the diagonal extensions intersect.
2. There is a single non P-vertex between p_1 and p_2, and the diagonal extensions do not intersect. In this case they must reach the same edge of P, as otherwise the rays would stop rotating when reaching that vertex.

3. There are two non P-vertices that lie on the same edge e of P. In this case both rays must intersect e as well, because they stop rotating when reaching a vertex of that edge.

In each case, we can extend the subchain by replacing it with the intersection point of the diagonal extensions in case 1, or by the intersection points of the diagonal extension with an edge of P (which also lies on a diagonal extension) in cases 2 and 3. In all cases, the edges of Q between p_1 and p_2 lie on diagonal extensions, and the vertices are from V_D.

The last step is exactly as in [16]: construct at most three convex polygons that contain Q''. Denote by $\hat{Q}$ the convex polygon whose vertices are all the P-vertices of Q''. Two polygons, Q_{even} and Q_{odd}, are constructed by adding to $\hat{Q}$ the even subchains for Q_{even}, and the odd subchains for Q_{odd} (see Figure 4.3). Notice that there must be an even number of subchains in each polygon, because connecting two consecutive P-vertices may result in a non-convex polygon.

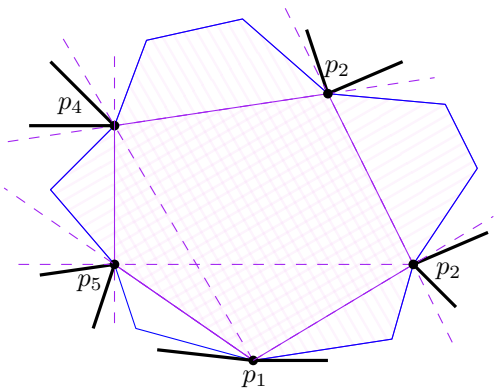


Figure 4.3: Decomposition of Q'' into two restricted polygons, Q_{even} and Q_{odd}, by alternating the subchains.

To prove the second part of the theorem, observe that Q'' can only have reflex vertices at reflex vertices of P, because the angles of the vertices on each subchain between two consecutive P-vertices are convex. If Q'' has a convex P-vertex p_1 (as in the figure above), then the angle at p_1 (that connects two such consecutive subchains) is also convex. Therefore, in this case, we can construct Q_{even} and Q_{odd} as above, but such that the remaining third polygon includes p_1. Then, since the angle to one of its neighbor subchains is convex, we can add it to the polygon that covers this subchain, and thus cover Q'' with at most two convex polygons. ■

We conclude with the following theorem.

Theorem 2 *There exists a subset $S^* \subseteq \mathcal{R}$ of restricted polygons such that S^* covers P (i.e. $\bigcup_{Q \in S^*} Q = P$), and $S^* \leq 3|OPT|$.*

5 Computing the largest uncovered convex polygon

Our main goal in this section is to solve the following problem.

Problem 3 (Largest Restricted Polygon) *Given a polygon P with n vertices, the set of faces $\mathcal{U}$ (as defined in Section 3.1), and a function $w : \mathcal{U} \to \{0, 1\}$, find a restricted polygon Q in P that maximizes $\sum_{f \in \mathcal{U}, f \subseteq Q} w(f)$.*

As mentioned in Section 3.2, in order for the solution to be utilized for the greedy MCC algorithm, we present an algorithm that gets the function w as an input, and computes the largest restricted polygon. The function w is then updated accordingly. Our algorithm is based on constructing a weighted DAG for each vertex of P, and then finding a heaviest path in the DAG.

A key insight in our construction is that it is sufficient to search for the largest restricted polygon within the visibility polygons of the vertices of P. This is because in the proof of Theorem 1, the construction of restricted polygons ensures that each such polygon contains at least one vertex of P. In other words, there exists a set of $3|OPT|$ restricted polygons that cover P, and each of them contains a vertex of P. We can therefore further restrict our set $\mathcal{R}$ to polygons that contain a vertex of P. Since restricted polygons are convex, if a restricted polygon Q contains a vertex v of P, then Q must lie entirely within the visibility polygon of v.

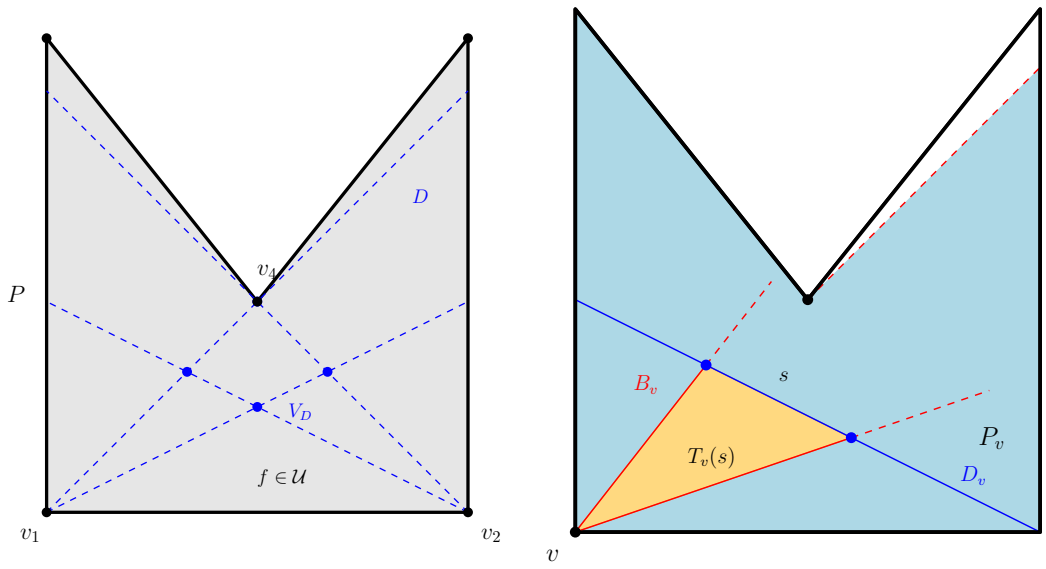
Observation 2 *Let Q be a restricted polygon that contains a vertex v of P, then $Q \subseteq Vis(v)$.*

Therefore, it is enough to iterate over all the vertices of P, and find for each vertex v the largest restricted polygon in $\mathrm{Vis}(v)$. Then we can take the maximum over all the restricted polygons that were found.

Notations overview and intuition. Before diving into the formal technical construction, it is helpful to review the core notations and the intuition behind them. The fundamental challenge of the Minimum Convex Cover problem is that the space of all possible convex polygons contained within P is infinite. To make this computationally tractable, the continuous space is discretized into a finite set of candidate shapes, limiting our search space.

Here is the logical progression of the notations used to build this discrete search space (see Figure 5.1):

- **The Polygon and Grid Lines (P, V, D, V_D):** We begin with our input polygon P and its vertices V. We extend lines between any two vertices of P to create a grid of maximal segments called diagonal extensions, denoted as D. The intersection points of these segments form our discrete candidate vertices, V_D.
- **The Atomic Regions ($\mathcal{U}$):** The arrangement of the segments in D cuts P into a finite set of indivisible polygonal faces, denoted as $\mathcal{U}$. Our algorithmic goal in each greedy step is to find a restricted convex polygon that covers the maximum number of uncovered faces in $\mathcal{U}$.
- **The Visibility Polygon of a Vertex v ($P_v = \mathbf{Vis}(v)$):** Because any valid restricted convex polygon must contain at least one vertex of P, it must lie entirely within the visibility polygon of that vertex. Therefore, we can narrow our search space to the visibility polygon of a specific vertex v, denoted P_v.
- **Refining the Grid inside P_v (D_v, B_v, C_v):** Inside P_v, we only care about the diagonal extensions that pass through it, forming the set D_v. We also draw rays from v to every grid intersection point (V_D) visible to it, creating visibility extensions, B_v. Together, the segments in D_v and B_v form a finer partition of cells inside P_v, called C_v.
- **Building the Graph ($T_v(s)$ and G):** To efficiently find the largest restricted polygon without brute-force checking, we model its perimeter as a path in a Directed Acyclic Graph, G. The vertices of G represent directed subsegments (s) of D_v. Connecting the vertex v to a subsegment s forms a subsegment triangle, $T_v(s)$. By finding the "heaviest" path in G, we are effectively finding a sequence of subsegments that enclose a convex polygon covering the most weight in $\mathcal{U}$.



(a) Global Discretization

(b) Local Visibility Space

Figure 5.1: Visualizing the discrete search space. **(a)** The global discretization of P showing diagonal extensions D (blue dashed), intersection vertices V_D, and an atomic face $f \in \mathcal{U}$. **(b)** The local search space from vertex v. The visibility polygon P_v is shaded light blue, with the white region indicating the area occluded by the reflex vertex. The visibility extensions B_v (red) connect v to points in V_D, enclosing the subsegment triangle $T_v(s)$ (orange) defined over a directed subsegment $s \in D_v$.

5.1 DAG construction for convex vertices

Fix a vertex v of P, and consider its visibility polygon $P_v = \text{Vis}(v)$. We assume in this section that v is a convex vertex (i.e., the internal angle at v is at most 180°).

Let $D_v = \{l \cap P_v\}_{l \in D}$ be the set of diagonal extensions restricted to P_v (see Figure 5.2). The segments in D_v are subsegments of the segments in D, and are maximal w.r.t. P_v. In other words, each segment in D_v has both its endpoints on ∂P_v (the boundary of P_v).

Directed subsegments. We now assign directions to the segments in D_v, as follows. For any segment $\overline{ab} \subseteq P_v$ with both endpoints a, b on ∂P_v, such that a appears before b when traversing the boundary in counterclockwise order from v, and $a \neq b \neq v$, the direction is from a to b, and we denote the directed segment by $\overrightarrow{ab}$. If, e.g., $a = v$, then the direction is $\overrightarrow{vb}$. For each directed segment $l = \overrightarrow{ab} \in D_v$, let $a = q_1, q_2, \ldots, q_t = b$ be the sequence of points from V_D on l, by their order along l. Every two consecutive points q_i, q_{i+1} define a ***directed subsegment*** of l.

Weighted cell Partition. Denote by B_v the set of extensions of segments $\overline{vp}$ such that $p \in V_D \cap P_v$ (see Figure 5.2). That is, B_v is the set of extensions of all the segments connecting v with a point from V_D in its visibility polygon. We call such an extension a ***visibility extension***. Notice that $|B_v| = O(n^4)$ because $|V_D| = O(n^4)$. Consider the set of cells C_v in P_v formed by the segments in $D_v \cup B_v$. This set of cells is a refinement of the arrangement $\mathcal{A}_D$, and we refer to it as the ***cell partition*** of P_v. Each cell $c \in C_v$ is contained in some face $f \in \mathcal{U}$. For a face $f \in \mathcal{U}$ that contains a set $C' \subseteq C_v$ of cells, we set the weight of each cell $c \in C'$ to be $w(c) = \frac{w(f)}{|C'|}$. That is, if $w(f) = 0$ then the weight of each cell c in the face f is 0, and otherwise it is one over the number of cells in f.

Notice that the segments in B_v have a common intersection point (at v), and each of them may intersect each of the segments in D_v. Because $|B_v| = O(n^4)$ and $|D_v| = O(n^2)$, the total number of intersection points of segments in $D_v \cup B_v$ is $O(n^6)$, and thus we have $|C_v| = O(n^6)$.

Subsegment triangles. For a directed subsegment $s = \overrightarrow{ab}$ of a segment in D_v, let $T_v(s) = \triangle vab$ be the ***subsegment triangle*** (SST) of s in P_v. Notice

that the extensions of $\overline{va}$ and $\overline{vb}$ are visibility extensions from B, because $a, b \in V_D$. Therefore, any such SST is a connected subset of faces from $\mathcal{U}$. Let $\mathcal{U}_v(s)$ be the set of faces from $\mathcal{U}$ that are contained in the SST $T_v(s)$. The weight of $T_v(s)$ is $w(T_v(s)) = \sum_{f \in U_v(s)} w(f)$.

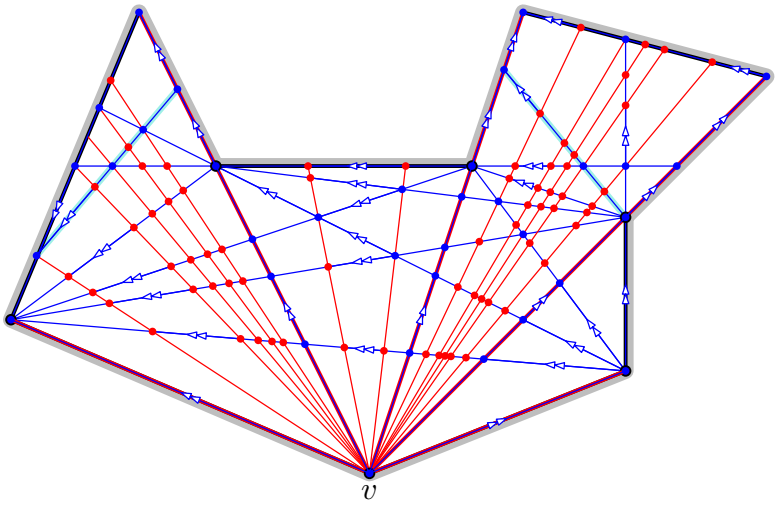


Figure 5.2: A visibility polygon P_v with the diagonal extensions in D_v (blue), and visibility extensions B_v (red). Two diagonal extensions are highlighted in turquoise, to show that diagonal extensions are coming from the entire polygon P. The blue points are the points of V_D, and the red points are the points of W. Each cell of the arrangement C_v is a connected union of faces from $\mathcal{U}$.

Using the above definitions, we construct a weighted directed acyclic graph $G = (V(G), E(G), w')$ as follows. First, for any connected union X of faces from $\mathcal{U}$, we denote $w(X) = \sum_{f \in \mathcal{U}, f \subseteq X} w(f)$. The set of vertices of G is of all directed subsegments in P_v, i.e.

$$V(G) = \{s \mid s \text{ is a directed subsegment of some } l \in D_v\}.$$

For a segment $s \in V(G)$, we let $w'(s) = w(T_v(s))$. There is a directed edge $e = (s_1, s_2)$ in $E(G)$ between two directed subsegments $s_1, s_2 \in V(G)$ if $s_1 = \overrightarrow{ab}$, $s_2 = \overrightarrow{bc}$, and c lies on the left side of the directed line through $\overrightarrow{ab}$. In other words, when walking from a to b and then to c we make a left-turn — the angle $\angle abc$ is at most 180° (see Figure 5.3). In the rest of the thesis, whenever we refer to the angle $\angle abc$ between two directed subsegments $\overrightarrow{ab}$, $\overrightarrow{bc}$, we mean the left-turn angle as defined here.

To show that G is a DAG, we first need the following observation.

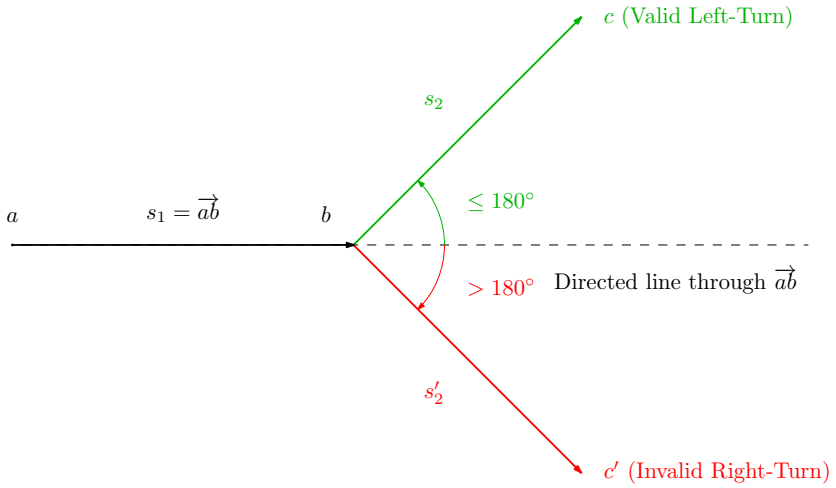


Figure 5.3: Defining the directed edges in $E(G)$. An edge exists from $s_1 = \overrightarrow{ab}$ to s_2 only if the subsequent subsegment makes a valid left-turn ($\leq 180°$). Subsegments that create a right-turn ($> 180°$) violate the convexity constraint and are not connected in the DAG.

Observation 3 *Let P be a simple polygon, and $s_1 = \overline{ab}$, $s_2 = \overline{cd}$ two maximal segments in P (i.e, their endpoints a, b, c, d are on ∂P). If s_1 and s_2 cross, then their counterclockwise order along the boundary of P is either a, c, b, d or a, d, b, c.*

Proof. The segment $s_1 = \overline{ab}$ splits the polygon into two sub polygons, and in order for the segments s_1 and s_2 to cross, the points c, d have to lie in different subpolygons. Therefore, when traversing ∂P in counterclockwise order from a, we first meet either c or d, and then b. ■

Claim 1 *The graph G defined above is a DAG.*

Proof. Assume by contradiction that G is not acyclic, so there exists a cycle $\pi = \{s_1, s_2, \ldots, s_k\} \in G$. First notice that the vertex v cannot participate in π, because it has no incoming edges. Denote the endpoints of the directed segment that defines the subsegment $s_i \in \pi$ by $\overrightarrow{a_i b_i}$. By the definition of G, every two consecutive directed segments $\overrightarrow{a_i b_i}$, $\overrightarrow{a_{i+1} b_{i+1}}$ cross each other. Since $P_v = \text{Vis}(v)$ is a simple polygon, by Observation 3 we get that their counterclockwise order along ∂P_v is $a_i, a_{i+1}, b_i, b_{i+1}$ ($a_i, b_{i+1}, b_i, a_{i+1}$ is not possible by the definition of $E(G)$, because then the direction of s_i, s_{i_1} would be toward their intersection point). Assume w.l.o.g. that a_2 is the first endpoint on ∂P_v after v. Then the order along ∂P_v is a_1, v, a_2, b_1, b_2, but this is in contradiction to the direction of $\overrightarrow{a_1 b_1}$. ■

Remark 1 *Note that in the above claim we did not use the fact that P_v is a visibility polygon, and interestingly, in fact G is a DAG for any simple polygon and a set of maximal segments.*

Next, we show how to construct G for the vertex v, given D and the arrangement $\mathcal{U}$. First notice that any two diagonal extensions that intersect are responsible for exactly four edges in $E(G)$, and thus $|E(G)| = O(n^4)$. Moreover, we can construct $V(G)$ and $E(G)$ in $O(n^4)$ time by using standard data structures for representing planar maps, such as DCEL [14].

The set B_v of visibility extensions can be computed in $O(n^4)$ time. Then, constructing the weighted cell partition C_v can be done in $O(n^6)$ time by sweeping the faces of the arrangement $\mathcal{U}$ according to the segments in B_v. During the sweeping process we also keep pointers between cells in C_v and their corresponding faces from $\mathcal{U}$. The weights of the cells in C_v can be computed afterwards by simply traversing the faces in $\mathcal{U}$. After each iteration of the greedy MCC algorithm (in which we add a restricted polygon Q to the cover), we need to update the weights of the cells in C_v. Recall that restricted polygons are connected unions of faces from $\mathcal{U}$. Whenever a restricted polygon Q is selected for the cover, we update an indicator for each of these faces, recording whether it has already been covered or not. This step takes $O(n^4)$ time, and then we can update the weight of each cell $c \in C_v$ in constant time by simply checking if its corresponding face from $\mathcal{U}$ was covered. Notice that in this case, the weight $w(c)$ becomes 0 (during the entire algorithm, the value of $w(c)$ changes only once).

Computing the weight function w' is slightly more complicated. There are $O(n^4)$ SSTs, and since each consists of a set of $O(n^6)$ cells from C_v, a trivial approach of iterating through the cells for each triangle would lead to an inefficient $O(n^{10})$ running time.

In Lemma 1 below, we show that given the weights of all cells $c \in C_v$, we can compute the weights of the SSTs in $O(n^6)$ time. This is done by introducing a two-phase approach based on a finer, non-overlapping partition of the SSTs:

- First, we define W as the set of intersection points between the diagonal extensions (D_v) and the visibility extensions (B_v) that are not already in V_D. These points naturally subdivide the large, overlapping SSTs into smaller, disjoint regions called Auxiliary Subsegment Triangles (ASTs). Because the ASTs are cleanly bounded by visibility rays emanating directly from v, they do not cross each other, allow-

ing us to compute their individual weights in a single $O(n^6)$ sweep.

- Second, we observe that every large SST is simply a concatenation of adjacent ASTs along a directed subsegment. Therefore, we can compute the weight of any SST by traversing its corresponding subsegment and maintaining a running sum of the AST weights.

Lemma 1 *Given a visibility polygon P_v and the cell partition C_v with a weight $w(c)$ for every cell $c \in C_v$, the weights of all the SSTs in P_v can be computed in $O(n^6)$ time in total.*

Proof. We define a set of auxiliary subsegment triangles, which is a refinement of the SSTs, as follows (see Figure 5.4). Let W be the set of vertices of the arrangement C_v that are not in V_D (see Figure 5.2). Each such point corresponds to an intersection point between a segment $l_1 \in D_v$ with a segment $l_2 \in B_v$ (by definition, the segments in B_v intersect only at v). The points in W subdivide the directed subsegments of each directed segment in D_v into auxiliary subsegment. Consider a directed segment $l = \overline{ab} \in D_v$, and denote the sequence of points from $V_D \cup W$ on l by $a = q_1, q_2, \ldots, q_k = b$. We add an auxiliary subsegment triangle (AST) $\triangle v q_i q_{i+1}$ for each auxiliary subsegment. Each such AST is a connected union of cells from C_v.

Consider the set of ASTs in the cone between two consecutive visibility extensions $l_1, l_2 \in B_v$. The auxiliary subsegments defining these ASTs are non-crossing (otherwise there would be a point from V_D between them, for which we add another visibility extension), and therefore the weights of the ASTs in this cone can be computed in time linear in their number, by accumulating the weights of cells in C_v in the order of containment between the ASTs.

After we have computed the weights of all ASTs, we can compute the weights of the SSTs as follows. Consider a directed segment $l = \overline{ab} \in D_v$ with a sequence of points $a = q_1, q_2, \ldots, q_k = b$ from $V_D \cup W$ as before. Notice that $q_1 \in V_D$ because l is maximal. Starting from q_3 (if $k = 2$, we are done), we iterate over the sequence of points in their order along l, and update the weights of their corresponding ASTs as follows: if $q_{i-1} \in W$, set $w(\triangle v q_{i-1} q_i) = w(\triangle v q_{i-1} q_i) + w(\triangle v q_{i-2} q_{i-1})$. In other words, we accumulate the weights of ASTs along l until we get to a point from V_D. The result of this process is that for a point $q_i \in V_D$ on l, the updated weight $w(\triangle v q_{i-1} q_i)$ will be the weight of the SST $\triangle v q_j q_i$, where q_j is the last point on l before q_i that belongs to V_D.

Overall, the running time of both steps is linear in the number of auxiliary subsegments, which is $O(n^6)$ (as the size of C_v).

■

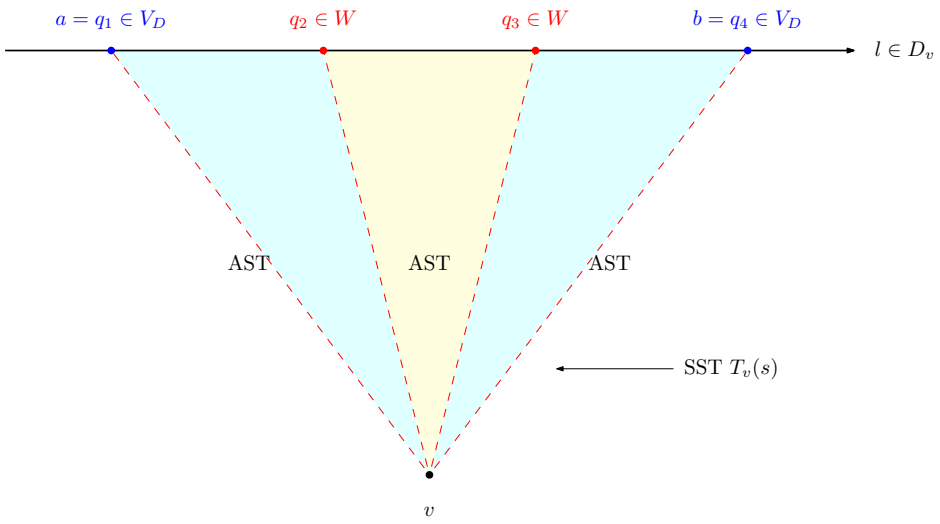


Figure 5.4: Subdividing a Subsegment Triangle (SST) into Auxiliary Subsegment Triangles (ASTs). The visibility extensions intersecting the directed segment l at points in W slice the larger, overlapping SST $T_v(s)$ into smaller, disjoint ASTs. This non-overlapping property allows for efficient linear-time weight computation.

We conclude that in each iteration of the algorithm, the total time for updating the weights of the SSTs for a single vertex v of P is $O(n^6)$, and thus the total running time for all vertices is $O(n^7)$.

5.2 The heaviest path in the DAG

Our goal now is to prove an equivalence between the heaviest path in G and a largest restricted polygon in P_v that contains v. Computing the heaviest path in a DAG G can be done in $O(|V(G)| + |E(G)|)$ time by sorting the vertices in topological order and then running a dynamic programming algorithm (see, e.g., [32]). Since $|V(G)| + |E(G)| = O(n^4)$, and constructing G takes $O(n^6)$ time, we have an algorithm with running time of $O(n^6)$ for computing the largest restricted polygon in P_v.

Below, we show an equivalence between maximal paths in G and restricted polygons in P_v that contain v as a vertex.

Claim 2 *Any maximal path in G starts with a directed subsegment $\overrightarrow{va}$ and ends with a directed subsegment $\overrightarrow{bc}$ such that $\overline{cv}$ lies on a diagonal extension, and the angle between $\overrightarrow{bc}$ and $\overrightarrow{cv}$ is at most $180°$.*

Proof. Consider a maximal directed path $\pi = s_1, s_2, \ldots, s_k$ in G. Since π is maximal, there is no directed subsegment s_0 such that there is an edge in G between s_0 and s_1. Denote $s_1 = \overrightarrow{xa}$, and assume by contradiction that $x \neq v$. If x is not on ∂P_v, then clearly there is a directed subsegment $s_0 = \overrightarrow{yx}$ on the same diagonal extension, and $\angle yxa = 180°$ so there is an edge between s_0 and s_1, a contradiction. Otherwise, x is on ∂P_v. If x is not a vertex of P, then by similar arguments we get a contradiction because any edge of P lies on a diagonal extension. If x is a convex vertex of P, then there must be a point y from V_D on one of its incident edges before x, and the angle between $\overrightarrow{yx}$ and $\overrightarrow{xa}$ is at most $180°$, so again we get a contradiction. If x is a reflex vertex, then consider the the diagonal extension of $\overline{vx}$ and let y be a point from V_D on $\overline{vx}$ before x. If the angle between $\overrightarrow{yx}$ and $\overrightarrow{xa}$ is larger than $180°$, then a is not visible from v. Again we get that there is an edge between $s_0 = \overrightarrow{yx}$ and $\overrightarrow{xa}$, a contradiction. We conclude that $s_1 = \overrightarrow{va}$.

Notice that by symmetric arguments, $\overrightarrow{cv}$ must lie on a diagonal extension, and the angle between $\overrightarrow{bc}$ and $\overrightarrow{cv}$ is at most $180°$. ■

For a maximal directed path $\pi = s_1, s_2, \ldots, s_k$ in G, let $Q(\pi) = \bigcup_{s \in \pi} T_v(s)$.

Claim 3 *For any maximal directed path π, $Q(\pi)$ is a restricted polygon that has v as one of its vertices.*

Proof. By the definition of G and Claim 2, if v is a convex vertex then the directed subsegments of π together with v enclose a convex polygon whose vertices are from V_D and whose edges are subsegments of diagonal extensions — a restricted polygon.

To see that $Q(\pi)$ is exactly the polygon enclosed by the directed subsegments of π, recall that each triangle $T_v(s_i)$ connects v to to consecutive vertices of V_D on π. In other words, the set $T_v(s_1), \ldots, T_v(s_k)$ is a triangulation of $Q(\pi)$. ■

Claim 4 *For any maximal restricted polygon Q that has v as one of its vertices, there exists a maximal directed path π such that $Q(\pi) = Q$.*

Proof. Since Q is a restricted polygon, each of its edges is a concatenation of subsegments in D_v (possibly a single subsegment). Denote $s_1, \ldots, s_k$ the chain of subsegments going counterclockwise from v along the border of Q. Each s_i has a vertex u_i in G. Since Q is a convex polygon, each angle between two consecutive subsegments s_i, s_{i+1} is at most 180° (a left turn). By the construction of G, there is an edge $e_i = (u_i, u_{i+1})$ in G. Thus, the directed path $\pi = e_1, \ldots, e_k$ exists in G. Notice that given a path $\sigma \in G$, $Q(\sigma)$ goes along the subsegments represented by the vertices of the path and then closes into a convex polygon using an extra edge connecting to v itself (connecting the end point of the segment represented by the last vertex in σ to the start point of the segment represented by the first vertex in σ). This is done since we wish for our G to be acyclic, thus, we add the last edge in $Q(\sigma)$ artificially without representing the respective connection using an edge in G. In the case of π, this is exactly the polygon Q. That is, $Q(\pi) = Q$ (as illustrated in Figure 5.5). ■

We obtain the following lemma.

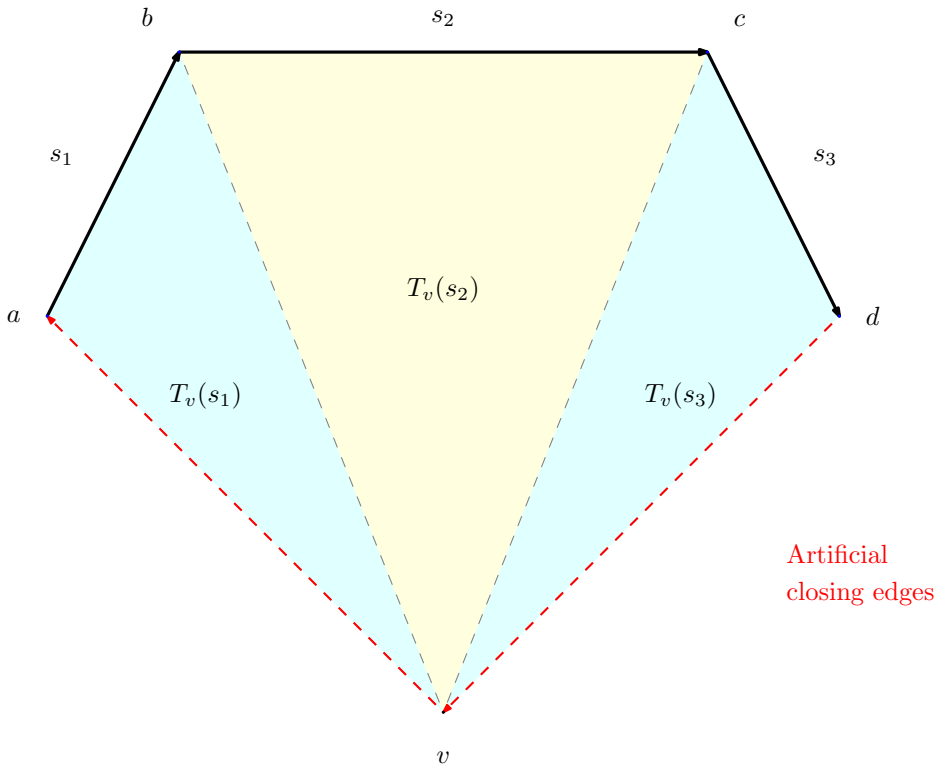
Lemma 2 *Let Q be a restricted polygon, and v a vertex in P. Then Q is a largest restricted polygon that contains v if and only if there exists a heaviest path π in G such that $Q(\pi) = Q$. Moreover, the weight of π is exactly $\sum_{f \in \mathcal{U}, f \subseteq Q} w(f)$.*

Proof. The first part of the lemma follows from Claim 3 and Claim 4. The weight of π is $\sum_{s \in \pi} w(T_v(s))$, and since $Q(\pi)$ is a restricted polygon, for every face $f \in \mathcal{U}$ it either contains all the cells from C_v in this face or does not contain any cell in this face. Since the sum of weights of the cells in a face f is equal to $w(f)$, we get that the weight of π is exactly $\sum_{f \in \mathcal{U}, f \subseteq Q} w(f)$. ■

We conclude that we can iterate over all vertices of P, and for each vertex v find a largest restricted polygon that contains v. Then, we can choose the restricted polygon Q that maximizes $\sum_{f \in \mathcal{U}, f \subseteq Q} w(f)$, as required. Since in this section we have only considered convex vertices, we now show how to handle reflex vertices.

5.3 Handling visibility polygons of reflex vertices

If v is a reflex vertex, we split P_v into two sub-polygons P_{v_1} and P_{v_2}, separated by one of the diagonal extensions from v (see Figure 5.6). Such a diagonal extension always exists by the extension of each of the edges of



Path $\pi = s_1 \to s_2 \to s_3$

$Q(\pi) = T_v(s_1) \cup T_v(s_2) \cup T_v(s_3)$

Figure 5.5: The equivalence between a maximal directed path π in the DAG and a restricted convex polygon $Q(\pi)$. The path $\pi = s_1, s_2, s_3$ defines the outer boundary, while the corresponding subsegment triangles $T_v(s_i)$ triangulate the interior. The polygon is geometrically closed by connecting the endpoints of the path back to the origin vertex v.

P which are adjacent in v. We then run the algorithm on each of them separately, and get two restricted polygons Q_1, Q_2.

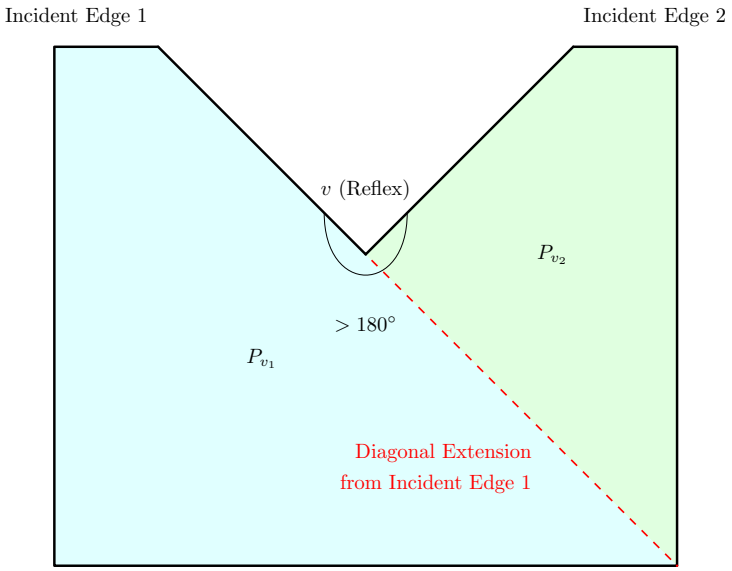


Figure 5.6: Handling a reflex vertex v. Because the internal angle at v exceeds $180°$, no single convex polygon can contain v without breaching the boundary. Extending one of the incident edges splits the visibility polygon P_v into two sub-polygons, P_{v_1} and P_{v_2}, ensuring v acts as a valid convex boundary point in both independent sub-problems.

The observation below follows from Lemma 2.

Observation 4 *Let Q^* be a largest restricted polygon in P_v that contains v, then $w(Q^*) = w((Q^* \cap P_{v_1}) \cup (Q^* \cap P_{v_2}))$ and thus $w(Q_1 \cup Q_2) \geq w(Q^*)$.*

To summarize, we build the DAGs for all the vertices of P. If a vertex v is a reflex vertex, we first split it into two polygons as described above, and it will have two DAGs associated with it. Constructing all the DAGs takes $O(n^7)$ time.

To find at most two restricted polygons $Q_1, Q_2 \in \mathcal{R}$ such that $w(Q_1 \cup Q_2) \geq w(Q^*)$, where Q^* is the restricted polygon that maximizes $w(Q^*)$, we find the heaviest path in each DAG (two DAGS, and two restricted polygons for a reflex vertex). Then we take the polygon $Q_1 = Q(\pi)$ (or two polygons Q_1, Q_2) that correspond to the heaviest path π (or paths), over all DAGs. By Lemma 2 and Observation 4, we obtain the following theorem.

Theorem 3 *Given the set of faces $\mathcal{U}$, with a binary function $w : \mathcal{U} \to \{0,1\}$, one can compute in $O(n^7)$ time at most two restricted polygons $Q_1, Q_2 \in \mathcal{R}$ such that $w(Q_1 \cup Q_2) \geq w(Q^*)$, where Q^* is the restricted polygon that maximizes $w(Q^*)$.*

6 The potato peeling problem

The core algorithmic challenge addressed in this thesis—identifying the optimal convex polygon to add to our cover—is a variation of a classic problem in computational geometry known as the *potato peeling problem*. Traditionally, this problem seeks the largest-area convex polygon contained within a given polygon P.

In the previous section, we solved a specific discrete variant which we term the **rotten meshed potato peeling problem**. In that variant, the "potato" (polygon P) is pre-partitioned into a "mesh" of regions (faces of $\mathcal{U}$). Some regions are "rotten" (already covered), and the goal is to find a convex polygon that covers the maximum number of "good" (uncovered) regions, while being allowed to overlap with the rotten ones.

While the "meshed" variant was constructed specifically for the purpose of the discrete MCC algorithm, the techniques we developed—specifically the DAG-based approach on visibility polygons—are robust. In this section, we first review the literature regarding the potato peeling problem. We then demonstrate the versatility of our approach by applying it to a more natural, continuous version of the potato peeling problem with "rotten" regions, yielding a constant-factor approximation algorithm.

6.1 Related work

The subproblem at the heart of our algorithm is a variant of the potato-peeling problem, also known as finding the "convex skull." Introduced by Goodman [20], this is a classic problem in computational geometry.

Problem 4 (Potato Peeling) *Given a polygon P, find a convex polygon of maximum area contained within it.*

The best-known exact algorithm for this problem runs in $O(n^7)$ time for

simple polygons and $O(n^8)$ for polygons with holes [8]. Due to this high complexity, approximation algorithms are of great interest. A notable result is a randomized $(1-\varepsilon)$-approximation algorithm that runs in near-linear time [7].

Other variants. Several variants of the potato peeling problem have been studied. Of particular relevance to our work is the **meshed potato peeling** problem, considered by Aronov et al. [3]. In this variant, the polygon P is given with a triangulation (a mesh), and the goal is to find a convex subpolygon of maximum *area* that is a union of triangles from the given mesh. They provide an exact $O(m^2)$ algorithm, where m is the number of triangles in the mesh.

While their problem setting appears similar to the subproblem we solved in Chapter 5, their algorithm is not applicable for two critical reasons. First, the objective functions differ fundamentally. Aronov et al. maximize geometric area, a property they leverage in their dynamic programming solution. In contrast, our objective in the previous section was to maximize the *number* of "good" regions covered, while allowing the solution to overlap with "rotten" regions at no cost. This non-standard objective invalidates the assumptions underpinning their method. Second, a direct application of their algorithm to our much finer mesh would be computationally infeasible.

Other variants include finding large simple shapes like triangles or ellipses inside a polygon [22] and discretized versions on lattice points [11], further highlighting the richness of this problem domain.

6.2 The rotten potato peeling problem

The rotten meshed potato peeling problem that we solved in Chapter 5 is rather restricted to the discretization required for the set cover framework. However, in this section we show that the method and geometric insights we developed can be utilized for approximating a more natural version of the problem, defined as follows.

Problem 5 (Rotten Potato Peeling) *Given a polygon P with n vertices, and a set R of polygons in P with a total of k vertices, find a convex polygon Q in P that maximizes the area of $Q \setminus R$.*

We provide a constant approximation to Problem 5 by solving a corresponding restricted version. We utilize the same general approach using DAGs as described in Section 5.1. The main difference lies in the weight function: here, there is no pre-calculated binary weight for mesh faces. Instead, we must define weights based on continuous area.

Below, we present a $\frac{1}{4}$-approximation algorithm for the rotten potato peeling problem that runs in $O(kn^5)$ time. First, observe that we can assume that an optimal solution Q^* to Problem 5 is a maximal polygon. We prove the following theorem regarding the existence of a good restricted approximation.

Theorem 4 *For any maximal convex polygon Q in P, there exists a restricted polygon Q' that contains a vertex v of P, such that $Area(Q' \setminus R) \geq \frac{1}{4} Area(Q \setminus R)$. Moreover, if v is a reflex vertex, split P into two polygons P_1, P_2 using one of the diagonal extensions from the edges incident to v. Then Q' is contained in one of P_1, P_2.*

Proof. Since Q is maximal, it must contain a vertex v of P (otherwise it can be expanded as in the proof of Theorem 1). Moreover, Q must lie entirely within the visibility polygon of v (because Q is convex).

We consider two cases based on the type of vertex v:

Case 1: v is a convex vertex. By Theorem 1, Q can be covered by at most two restricted polygons Q_1, Q_2, and by the construction, both of them have v as a vertex. Clearly, the union of these two covers Q, so the sum of their valid areas must be at least the valid area of Q. By the pigeonhole principle, one of Q_1, Q_2 must satisfy:

$$\text{Area}(Q_i \setminus R) \geq \frac{1}{2}\text{Area}(Q \setminus R).$$

Case 2: v is a reflex vertex. Split P into two polygons P_1, P_2 using one of the diagonal extensions from the edges incident to v. Denote $Q_1 = Q \cap P_1$ and $Q_2 = Q \cap P_2$. Since $Q = Q_1 \cup Q_2$ (and they are disjoint except for the boundary), one of them must contain at least half the area. Assume without loss of generality that $\text{Area}(Q_1 \setminus R) \geq \frac{1}{2}\text{Area}(Q \setminus R)$.

Now, apply Theorem 1 on the polygon P_1 and the convex shape Q_1. In P_1, v behaves as a convex vertex (or part of the boundary). Thus, there exist two restricted polygons Q_3, Q_4 in P_1, both having v as a vertex, that together cover Q_1. Consequently, one of them must capture at least half of

the remaining area of Q_1:

$$\max\{\text{Area}(Q_3 \setminus R), \text{Area}(Q_4 \setminus R)\} \geq \frac{1}{2}\text{Area}(Q_1 \setminus R).$$

Combining this with our earlier bound for Q_1:

$$\frac{1}{2}\text{Area}(Q_1 \setminus R) \geq \frac{1}{2}\left(\frac{1}{2}\text{Area}(Q \setminus R)\right) = \frac{1}{4}\text{Area}(Q \setminus R).$$

Thus, there exists a restricted polygon Q' satisfying the theorem statement. ■

Algorithm and complexity. We construct a DAG for each convex vertex v of P, exactly as in Chapter 6, except for the weight function. We define the weight of each vertex (which represents a directed subsegment s) in the DAG as:

$$w'(s) = \text{Area}(T_v(s) \setminus R).$$

In other words, the weight of s is the area of the "good" regions that the subsegment triangle $T_v(s)$ covers. We compute this weight by intersecting the triangle $T_v(s)$ with the set R. This geometric intersection takes $O(|R|) = O(k)$ time per triangle. Since there are $O(n^5)$ total subsegment triangles (SSTs) across all DAGs, the total weight computation takes $O(kn^5)$.

For each reflex vertex v of P, we split P into two polygons P_1, P_2 using a diagonal extension as described in Theorem 4. We then construct a DAG for each of P_1 and P_2 separately, treating v as a convex vertex within those sub-problems.

To find the solution, we compute the heaviest path in all constructed DAGs. For a path π in the graph, we define the polygon $Q(\pi)$ as before. Since the SSTs corresponding to the vertices of π form a triangulation of $Q(\pi)$, the area of the "good" regions that $Q(\pi)$ covers is exactly the sum of the weights of the vertices in the path.

By the same logic established in Chapter 5, finding the heaviest path in the DAG is equivalent to finding the restricted polygon rooted at v that maximizes the uncovered area. We iterate over all vertices of P (splitting reflex vertices as necessary) and return the maximum over all computed paths.

This algorithm finds, in $O(kn^5)$ time, a restricted polygon Q that satisfies the properties of Theorem 4. Since the theorem guarantees this polygon

captures at least a quarter of the optimal area, we obtain the following result.

Theorem 5 *Given a polygon P with n vertices, and a set R of polygons in P with a total of k vertices, there is an algorithm that in $O(kn^5)$ time finds a convex polygon Q in P such that $Area(Q \setminus R) \geq \frac{1}{4} Area(Q^* \setminus R)$, where Q^* is a convex polygon that maximizes $Area(Q^* \setminus R)$.*

7 Heuristics and experimental evaluation

While the algorithm presented in Chapter 5 achieves an asymptotic approximation ratio of $O(\log n)$ with a polynomial running time of $O(n^8)$, the high degree of the polynomial makes it impractical for large-scale instances. In real-world applications and competitive settings, such as the CG:SHOP Challenge 2023 [17], efficiency is often paramount.

In this section, we evaluate practical heuristic algorithms for the Minimum Convex Cover problem. These methods relax the rigorous grid constraints of the theoretical approach, relying instead on polygon triangulation and randomized greedy strategies. We evaluate these algorithms on a diverse benchmark dataset, comparing their performance against each other, against the recent theoretical 6-approximation algorithm for simple polygons [6], and against the state-of-the-art solvers from the competition.

7.1 Triangulation-based heuristics

Our heuristic approach simplifies the search space by utilizing a constrained decomposition of P. Instead of the dense arrangement $\mathcal{U}$ (size $O(n^{10})$), we start with a triangulation of P, denoted as $\mathcal{T}$, where $|\mathcal{T}| = O(n)$. We developed two primary algorithms based on this structure.

7.1.1 Heuristic A: randomized edge removal

This algorithm employs a local search strategy to merge triangles into larger convex polygons.

1. **Initialization:** Compute a triangulation $\mathcal{T}$ of P.

2. **Iterative Merging:** Iterate through the internal edges of the triangulation in a random order. For each edge e shared by two polygons P_a and P_b, check if the union $P_{new} = P_a \cup P_b$ is convex.
3. **Update:** If P_{new} is convex, remove e, replace P_a and P_b with P_{new}, and continue.

This method is computationally lightweight but susceptible to local optima, as the specific order of edge removal dictates the final configuration.

7.1.2 Heuristic B: greedy dual-graph expansion

This algorithm, which proved to be our most effective method, can be viewed as a heuristic implementation of the *Rotten Meshed Potato Peeling* framework established in Chapter 5. Instead of searching the exact DAG of visibility subsegments, we perform a greedy search over the dual graph of the triangulation.

1. **Selection:** Maintain a list of uncovered triangles. Select a random uncovered triangle $t \in \mathcal{T}$ as the "seed".
2. **Expansion:** Iteratively expand the seed polygon by merging it with neighboring triangles, provided the union remains convex. We employ a limited-depth search to find a locally maximal convex polygon Q starting from t.
3. **Update:** Add Q to the solution set, mark all triangles contained in Q as covered, and repeat until P is fully covered.

To manage complexity for large instances ($n > 10,000$), we implemented geometric hashing to cache convexity checks and pruning techniques to terminate search branches that fail to improve the cover size after a fixed number of iterations.

7.2 Implementation and benchmarks

We implemented these algorithms in Python using the `shapely` library for geometric predicates and `tinygeom2d` for visibility calculations. We evaluated our approach using the benchmark set from the **CG:SHOP Challenge 2023** [17], consisting of 206 instances divided into categories such

as "Cheese" (polygons with many holes), "Maze" (orthogonal corridors), and "SRPG" (random perturbations).

Our implementation of Heuristic B achieved significant success in the competition, ranking **8th overall** and **4th in the Junior category** out of 21 participating teams.

7.3 Experimental results

Heuristics comparison. Heuristic B (Greedy Expansion) consistently outperformed Heuristic A (Edge Removal). On complex instances like the "Maze" dataset, Heuristic B produced covers that were on average **28% smaller** than those produced by Heuristic A. While Heuristic B is computationally heavier—approximately 90% slower on the largest inputs—the trade-off is justified by the significantly higher quality of the solution.

Heuristics vs. theoretical bounds. A key finding of our experiments is the gap between theoretical worst-case guarantees and practical average-case performance. We compared Heuristic B against the 6-approximation algorithm for simple polygons [6] on the *Weakly Visible* instances from the challenge.

As illustrated in Figure 7.1, Heuristic B produced solutions that were, on average, **104% better** (i.e., less than half the cardinality) than the output of the 6-approximation algorithm. This discrepancy arises because the theoretical algorithm adds unnecessary polygons to satisfy worst-case structural lemmas, whereas our greedy heuristic adapts to the local geometry.

7.3.1 Comparison with the state-of-the-art

To contextualize our results, we compared our greedy approach against the winning teams of the CG:SHOP Challenge.

The winning approaches. The competition was dominated by teams utilizing heavy combinatorial optimization:

- **Team DIKU (1st Place):** Abrahamsen et al. [2] employed a method based on **Vertex Clique Covers**. They constructed a visibility graph

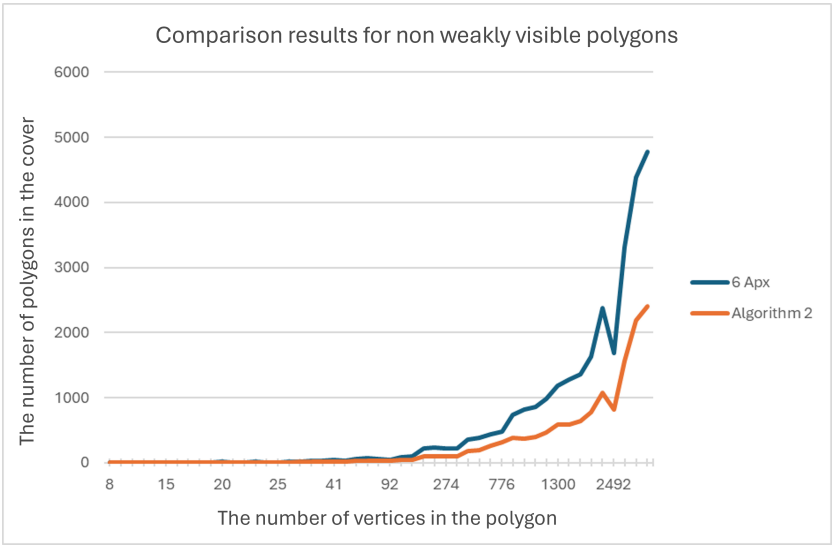


Figure 7.1: Comparison of solution sizes: Heuristic B (Orange) vs. the Theoretical 6-Approximation (Blue). The heuristic consistently finds significantly smaller covers (lower is better).

where cliques correspond to convex polygons and solved the NP-hard Minimum Clique Cover problem on this graph.

- **Team Shadoks (2nd Place):** Fonseca [13] used a "generate-and-cover" strategy similar to ours but formulated the selection phase as a Set Cover problem solved via Integer Linear Programming (ILP).

Comparative analysis. While the rigorous optimization of Team DIKU yielded better approximation ratios on the hardest instances, our Heuristic B proved to be a competitive alternative. Our greedy approach is significantly faster than the ILP-based solvers and clique-cover methods, making it highly suitable for real-time applications. The fact that a randomized greedy heuristic ranked 8th overall confirms that the "Potato Peeling" strategy—when coupled with effective heuristics—is a viable path for practical MCC solvers.

8 Conclusions and future work

In this thesis, we have presented a comprehensive study of the Minimum Convex Cover problem. Our main contributions include a novel $O(\log n)$-approximation algorithm for polygons with holes using a DAG-based greedy strategy, a significantly faster polynomial runtime ($O(n^8)$) compared to previous methods ($O(n^{29})$), and a new $\frac{1}{4}$-approximation for the "Rotten Potato Peeling" problem with forbidden regions. We also demonstrated the practical viability of these concepts through heuristic implementations that performed competitively in the CG:SHOP Challenge.

While our work bridges a significant gap in the complexity of MCC, several intriguing open problems and research directions remain.

Bounded aspect ratio and submodularity. One promising direction is to explore the problem under geometric constraints, such as requiring the covering convex polygons to have a *bounded aspect ratio* (i.e., "fat" polygons). In many practical applications, long and thin slivers are undesirable. The function defining the area covered by a union of shapes is naturally submodular. If we restrict the dictionary of allowed shapes to those with bounded aspect ratio, it may be possible to leverage stronger properties of submodular optimization. Specifically, it would be interesting to investigate if the submodular set cover approach can be applied here. Such an approach might not only improve the approximation factor beyond $O(\log n)$ for this subclass of shapes but could also potentially lead to faster, randomized greedy strategies that avoid the explicit construction of the full arrangement $\mathcal{U}$.

Further reduction of runtime. While $O(n^8)$ is a major improvement over $O(n^{29})$, it remains a high-degree polynomial. A significant portion of this complexity comes from the size of the universe $\mathcal{U}$ defined by the arrangement of visibility extensions. An open question is whether a coarser dis-

cretization exists that still preserves the constant-factor approximation for the restricted convex cover. Finding a "sparse" set of candidate restricted polygons could potentially lower the runtime to $O(n^6)$ or even less.

Fixed-parameter tractability. The general MCC problem is NP-hard and APX-hard. However, the complexity might be manageable with respect to the size of the optimal solution, k. Investigating whether the problem is Fixed-Parameter Tractable (FPT) for general polygons with holes is a natural next step. While FPT results exist for covering points, extending these to covering the entire continuous domain of a polygon with holes remains a challenge.

Heuristic guarantees. In Chapter 7, we demonstrated that heuristics based on triangulation and random edge removal perform well in practice (e.g., in the CG:SHOP challenge). Bridging the gap between these practical heuristics and theoretical guarantees is desirable. For instance, can we prove an approximation ratio for the triangulation-expansion heuristic (Algorithm 2) if we assume the polygon P has certain properties, such as being orthogonal or having a bounded feature size?

Refining the potato peeling approximation. In Chapter 6, we provided a $\frac{1}{4}$-approximation for the Rotten Potato Peeling problem. It is currently unknown if this factor is tight for restricted polygons. Improving this constant, or finding a Polynomial Time Approximation Scheme (PTAS) for the potato peeling variant with forbidden regions, would directly improve the constants hidden in our MCC approximation.

Bibliography

[1] ABRAHAMSEN, M. Covering polygons is even harder. In *Proceedings of the 62nd IEEE Symposium on Foundations of Computer Science (FOCS)* (2021), IEEE Computer Society, pp. 124–135.

[2] ABRAHAMSEN, M., BILLE MEYLING, W., AND NUSSER, A. Constructing concise convex covers via clique covers (cg challenge). In *39th International Symposium on Computational Geometry (SoCG 2023)* (2023), Schloss Dagstuhl–Leibniz-Zentrum für Informatik, pp. 66–1.

[3] ARONOV, B., VAN KREVELD, M., LÖFFLER, M., AND SILVEIRA, R. I. Peeling meshed potatoes. *Algorithmica 60*, 2 (2011), 349–367.

[4] BANDYAPADHYAY, P., BHOWMICK, D., AND VARADARAJAN, K. A ptas for the minimum convex partition of a polygonal domain. In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA)* (2021), SIAM, pp. 2828–2847.

[5] BRÖNNIMANN, H., AND GOODRICH, M. T. Almost optimal set covers in finite vc-dimension. In *Proceedings of the eleventh annual symposium on Computational geometry* (1995), pp. 293–302.

[6] BROWNE, R., KASTHURIRANGAN, P. N., MITCHELL, J. S., AND POLISHCHUK, V. Constant-factor approximation algorithms for convex cover and hidden set in a simple polygon. In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)* (2023), IEEE, pp. 1357–1365.

[7] CABELLO, S., CIBULKA, J., KYNČL, J., SAUMELL, M., AND VALTR, P. Peeling potatoes near-optimally in near-linear time. In *Proceedings of the thirtieth annual symposium on Computational geometry* (2014), pp. 224–231.

[8] CHANG, J.-S., AND YAP, C.-K. A polynomial solution for the potato-peeling problem. *Discrete & Computational Geometry 1*, 2 (1986), 155–182.

[9] CHAZELLE, B. Triangulating a simple polygon in linear time. *Discrete & Computational Geometry 6*, 5 (1991), 485–524.

[10] CHAZELLE, B., AND DOBKIN, D. P. Optimal convex decompositions. In *Computational Geometry*, G. T. Toussaint, Ed., vol. 2 of *Machine Intelligence and Pattern Recognition*. North-Holland, 1985, pp. 63–133.

[11] CROMBEZ, L., DA FONSECA, G. D., AND GÉRARD, Y. Peeling digital potatoes. *arXiv preprint arXiv:1812.05410* (2018).

[12] CULBERSON, J., AND RECKHOW, R. Covering polygons is hard. In *[Proceedings 1988] 29th Annual Symposium on Foundations of Computer Science* (1988), pp. 601–611.

[13] DA FONSECA, G. D. Shadoks approach to convex covering. *arXiv preprint arXiv:2303.07696* (2023).

[14] DE BERG, M., CHEONG, O., VAN KREVELD, M., AND OVERMARS, M. *Computational geometry: algorithms and applications*. Springer, 2008.

[15] EIDENBENZ, S., STAMM, C., AND WIDMAYER, P. Inapproximability results for guarding polygons and terrains. *Algorithmica 31*, 1 (2001), 79–113.

[16] EIDENBENZ, S., AND WIDMAYER, P. An approximation algorithm for minimum convex cover with logarithmic performance guarantee. In *European Symposium on Algorithms* (2001), Springer, pp. 333–344.

[17] FEKETE, S. P., KELDENICH, P., KRUPKE, D., AND SCHIRRA, S. Minimum coverage by convex polygons: The cg: Shop challenge 2023. *arXiv preprint arXiv:2303.07007* (2023).

[18] FILTSER, O., MAIMON, T., AND YOMTOVYAN, O. Peeling rotten potatoes for a faster convex cover approximation. In *Proceedings of the 2026 ACM-SIAM Symposium on Discrete Algorithms (SODA 2026)* (2026). To appear.

[19] FOWLER, R. J., PATERSON, M. S., AND TANIMOTO, S. L. Optimal packing and covering in the plane are np-complete. *Information processing letters 12*, 3 (1981), 133–137.

[20] GOODMAN, J. E. On the largest convex polygon contained in a non-convex n-gon, or how to peel a potato. *Geometriae Dedicata 11*, 1 (1981), 99–106.

[21] GREENE, D. H. The decomposition of a polygon into convex parts. In *Computational Geometry*, vol. 1. JAI Press, 1983, pp. 235–259.

[22] HALL-HOLT, O., KATZ, M. J., KUMAR, P., MITCHELL, J. S., AND SITYON, A. Finding large sticks and potatoes in polygons. In *Soda* (2006), vol. 6, pp. 474–483.

[23] HOCHBAUM, D. S., AND MAASS, W. Approximation schemes for covering and packing problems in image processing and vlsi. *Journal of the ACM (JACM) 32*, 1 (1985), 130–136.

[24] KEIL, J. M. Polygon decomposition. In *Handbook of Computational Geometry*, J.-R. Sack and J. Urrutia, Eds. Elsevier, 2000, pp. 491–518.

[25] KEIL, J. M., AND SNOEYINK, J. On the time bound for convex decomposition of simple polygons. *International Journal of Computational Geometry & Applications 12*, 03 (2002), 181–192.

[26] LEE, D.-T., AND LIN, A. Computational complexity of art gallery problems. *IEEE Transactions on Information Theory 32*, 2 (1986), 276–282.

[27] LIEN, J.-M., AND AMATO, N. M. Approximate convex decomposition of polygons. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry* (2004), SCG '04', pp. 17–26.

[28] LINGAS, A. The power of non-rectilinear holes. In *Automata, Languages and Programming: Ninth International Colloquium, Aarhus, Denmark, July 12–16, 1982, Proceedings 9* (1982), Springer, pp. 369–383.

[29] LINGAS, A., PINTER, R. Y., RIVEST, R. L., AND SHAMIR, A. Minimum edge length partitioning of rectilinear polygons. *Algorithmica 5* (1982), 507–538.

[30] MEGIDDO, N., AND TAMIR, A. On the complexity of locating linear facilities in the plane. *Operations Research Letters 1*, 5 (1982), 194–197.

[31] MITCHELL, J. S. Geometric set cover and hitting set for combinations of objects. In *34th International Symposium on Computational Geometry (SoCG 2018)* (2018).

[32] NOLTEMEIER, H. An algorithm for the determination of longest distances in a graph. *Mathematical Programming 9* (1975), 350–357.

[33] O'ROURKE, J. *Art Gallery Theorems and Algorithms*. Oxford University Press, Inc., USA, 1987.

[34] O'ROURKE, J., AND SUPOWIT, K. Some np-hard polygon decomposition problems. *IEEE Transactions on Information Theory 29*, 2 (1983), 181–190.

[35] PAVLIDIS, T. Analysis of set patterns. *Pattern Recognition 1*, 2 (1968), 165–178.

[36] SHERMER, T. C. Recent results in art galleries. *Proceedings of the IEEE 80*, 9 (1992), 1384–1399.

תקציר

הבעיה של מציאת כיסוי קמור מינימלי עבור מצולע P --- זיהוי הקבוצה הקטנה ביותר של מצולעים קמורים המוכלים ב־P שאיחודם מכסה את P במלואו --- היא בעיה יסודית בגיאומטריה חישובית, אשר הוכחה כבעיה שלמה במחלקת הסיבוכיות $\exists\mathbb{R}$. במשך למעלה משני עשורים, התוצאה התיאורטית הטובה ביותר הייתה אלגוריתם קירוב מסדר $O(\log n)$ שפותח על ידי איידנבנץ ווידמאייר (2001); עם זאת, זמן הריצה שלו, $O(n^{29}\log n)$, הופך אותו ללא מעשי.

תזה זו מציגה מסגרת אלגוריתמית חדשה המשמרת את יחס הקירוב $O(\log n)$ תוך הפחתה דרסטית של סיבוכיות הזמן ל־$O(n^8)$. אנו משיגים זאת על ידי אופטימיזציה של שלב הבחירה החמדנית, ומידולו כבעיית "קילוף תפוח אדמה רקוב ומרושת". בגרסה זו, אנו מחפשים מצולע קמור הממקסם את הכיסוי של אזורים שטרם כוסו ("טובים") תוך התעלמות מאזורים שכבר כוסו ("רקובים"). החידוש העיקרי שלנו כרוך בבניית גרפים מכוונים חסרי מעגלים על פוליגוני הנראות של קודקודי P. טרנספורמציה זו מצמצמת את בעיית האופטימיזציה הגיאומטרית למציאת מסלול בעל משקל מקסימלי בתוך הגרף, ומאפשרת זיהוי יעיל של החלק הקמור האופטימלי.

מעבר למקרה הבדיד, בפרק 6 אנו מדגימים את הגמישות של הגישה שלנו על ידי טיפול בבעיית "קילוף תפוח אדמה רקוב" הרציפה. במקרה זה, בהינתן קבוצה של מצולעים "רקובים" R בתוך P, אנו מחפשים מצולע קמור $Q \subseteq P$ הממקסם את השטח של $Q \setminus R$. אנו מציגים אלגוריתם המחשב קירוב בשיעור $\frac{1}{4}$ עבור גרסה זו בזמן $O(kn^5)$, כאשר k הוא מספר הקודקודים של המצולעים הרקובים. לבסוף, בפרק 7, אנו מציעים יוריסטיקות מעשיות המציגות ביצועים טובים באופן אמפירי, ובפרק 8 אנו דנים בכיוונים אפשריים למחקר עתידי.



האוניברסיטה הפתוחה
המחלקה למתמטיקה ולמדעי המחשב

אלגוריתמי קירוב לבעיית הכיסוי בפוליגונים קמורים

עבודת תזה זו הוגשה כחלק מהדרישות לקבלת תואר
"מוסמך למדעים" M.Sc. במדעי המחשב
באוניברסיטה הפתוחה
המחלקה למתמטיקה ומדעי המחשב

על־ידי
אופיר יומטוביאן

בהנחיית ד"ר עומרית פילצר

אוקטובר 2025

תוכן עניינים

1 מבוא 1

2 רקע 4
2.1 בעיות כיסוי גיאומטריות 4
2.2 בעיות חלוקה גיאומטריות 6
2.3 בעיות רלוונטיות נוספות 8

3 עבודות קודמות 9
3.1 סימונים והגדרת הבעיה 9

4 דיסקריטיזציה של הבעיה 12
4.1 ניסוח ככיסוי קבוצות מינימלי 12
4.2 האלגוריתם של איידנבנץ ווידמאייר 13
4.3 כיסוי בקמורים מוגבל 15

5 חישוב המצולע הגדול ביותר שטרם כוסה 19
5.1 בניית גרף מכוון ללא מעגלים לקודוקדים קמורים 22
5.2 המסלול המקסימלי בגרף 27
5.3 טיפול בפוליגוני נראות לקודקודים קעורים 29

6 בעיית קילוף תפוח אדמה 33
6.1 עבודות קודמות 33
6.2 בעיית קילוף תפוח אדמה רקוב ומרושת 34

7 יוריסטיקות והערכה אמפירית 38
7.1 יוריסטיקות מבוססות חלוקה למשולשים 38
7.1.1 יוריסטיקה א: הסרת קשתות רנדונמליות 38
7.1.2 יוריסטיקה ב: הרחבה בצורה חמדנית 39
7.2 מימוש ומדדי תוצאות 39
7.3 בדיקות אמפיריות 40
7.3.1 השוואה עם הטכנולוגיה המקתדמת ביותר 40

8 סיכום 42